

Harness Engineering 约束文件

数字人网页导航助手 — AI Coding Agent 驾驭工程蓝图

一、什么是 Harness Engineering

Harness Engineering（驾驭工程）是 OpenAI 于 2026 年 2 月正式提出的概念，核心理念是：

与其预测 AI 会犯什么错误，不如设计一个让错误难以发生的环境。

它不是 Prompt Engineering（2023-2024），也不是 Context Engineering（2025 中期），而是更上一层的完整约束体系 — 包括架构约束、反馈回路、文档规范、linting 规则和生命周期管理。

关键数据点：

- LangChain 的 coding agent 仅通过改进 harness（不换模型），在 Terminal Bench 2.0 上从 52.8% 跃升至 66.5%（Top 30 → Top 5）
- OpenAI 三人团队用 harness 方法在 1/10 的时间内构建了 100 万行代码的产品
- 行业共识：***"交付最快的团队不是在用更好的模型，而是在用更好的 harness"***

二、核心原则

原则 1：让错误难以发生（Make Mistakes Hard）

不是靠 AI 的自觉，而是靠结构性约束。如果一个模块不应该访问 DOM，它就不应该能 import DOM 相关的 API。如果 API key 不能硬编码，就应该有 pre-commit hook 拦截。

本项目的实施：

- 每个 JS 文件有明确的作用域边界
- Content Script 只能操作 DOM，不能调 API
- Background SW 只能调 API 和做编排，不能操作 DOM
- Side Panel 只负责渲染和语音，不做业务逻辑

原则 2：Plan-Execute-Verify（PEV）

打破"复合错误级联"（Compounding Error Cascade）。AI agent 不擅长给自己的作业打分，所以生成和评估必须是两个独立的步骤。

本项目的实施：

- Plan：每个功能模块先写伪代码和接口定义
- Execute：严格按照接口实现
- Verify：用独立的验证脚本测试，不依赖开发者自觉

原则 3：每个会话只做一件事（One Task Per Session）

这是"防止失败最多的单条规则"。多任务并行在同一会话中会增加错误级联风险。

本项目的实施：

- 每个 JS 模块单独一个 session 开发
- 模块间通过明确的接口契约连接
- 不允许多个模块在同一轮对话中混改

原则 4：结构化文件优于上下文依赖

依赖 AI 的上下文记忆是不可靠的。关键信息必须写入文件，机器可读。

本项目的实施：

- AGENTS.md 文件定义每个子系统的规则
- JSON 格式的任务列表（append-only）
- 接口契约用 JSDoc 注解，不是注释

原则 5：架构约束通过工具强制执行

不是"请遵守规则"，而是"违反规则会报错"。Lint 的错误信息不仅标记违规，还告诉 agent 如何修复。

本项目的实施：

- ESLint 配置针对本项目定制规则

- 模块间 import 关系受约束
- pre-commit hook 检查 API key 泄漏、文件大小限制

三、模块边界与契约

3.1 模块划分

```
extension/
├─ lib/
│   ├─ dom-distiller.js    → 纯函数，输入DOM树，输出蒸馏JSON
│   ├─ selector-engine.js  → 纯函数，输入selector+DOM，输出element|null
│   ├─ task-queue.js       → 状态机，输入任务数组，输出执行结果
│   ├─ tts.js              → 副作用函数，输入文本，输出音频流
│   └─ llm-client.js       → 副作用函数，输入prompt，输出LLM响应
├─ content/
│   ├─ content.js          → 编排上述lib模块，处理页面交互
│   └─ content.css         → 高亮动画样式
├─ background/
│   └─ service-worker.js   → 消息路由 + API调用 + 状态管理
└─ sidepanel/
    ├─ sidepanel.html
    └─ sidepanel.js        → 3D渲染 + ASR拾音 + TTS播放
```

3.2 模块契约（JSDoc 接口定义）

dom-distiller.js

```
/**
 * @typedef {Object} DistilledElement
 * @property {string} tag - HTML tag name
 * @property {string} text - Visible text content
 * @property {string} weakSelector - Simple class-based selector
 * @property {string} strongSelector - Full attribute-based selector
 * @property {string} textFallback - XPath fallback for text matching
 * @property {string} region - Page region (header/nav/main/aside/footer)
 * @property {Object} rect - BoundingBoxClientRect {x, y, w, h}
 */

/**
 * @param {Document} dom - The page document
 * @returns {{ regions: Object, elements: DistilledElement[], summary:
string }}
 */
function distill(dom) { ... }
```

selector-engine.js

```

/**
 * @typedef {'exact'|'weak'|'xpath'|'retry'|'verbal'} FallbackLevel
 */

/**
 * @param {string} selector - Target CSS selector
 * @param {string} fallbackText - Text content for XPath fallback
 * @returns {{ element: Element|null, level: FallbackLevel }}
 */
function resolve(selector, fallbackText) { ... }

```

llm-client.js

```

/**
 * @param {string} pageSummary - Distilled page structure
 * @param {string} userIntent - User's spoken intent
 * @param {'deepseek-chat'|'deepseek-v4-flash'} model
 * @returns {Promise<{ target: string, action: string, speech: string,
tasks?: Array }>}}
 */
async function infer(pageSummary, userIntent, model) { ... }

```

3.3 禁止事项

禁止行为	原因
Content Script 直接调用 fetch API	应该通过 Background SW 代理
Background SW 直接操作 DOM	Background SW 没有 DOM 访问权限
硬编码 API Key	必须通过 chrome.storage 管理
模块间循环引用	会导致不可预测的初始化顺序
在 content.css 中使用 !important	会污染宿主页面样式
超过 300 行的单文件	超过此阈值需要拆分

四、通信协议规范

4.1 Chrome Runtime Messages 契约

所有跨进程通信必须使用 `chrome.runtime.sendMessage`，消息格式为：

```
{
  type: string,      // 消息类型，必须是枚举值之一
  payload: any,      // 消息载荷，类型由 type 决定
  id: string,        // 唯一消息ID，用于请求-响应配对
  timestamp: number // 发送时间戳
}
```

4.2 消息类型枚举

type	方向	payload	响应
DOM_DISTILL	BG→CS	无	{ regions, elements, summary }
EXEC_HIGHLIGHT	BG→CS	{ selector, animation }	{ success, element? }
EXEC_CLICK	BG→CS	{ selector, fallbackText }	{ success, level }
EXEC_TASK_QUEUE	BG→CS	{ tasks: Task[] }	{ completed, failed? }
ASR_RESULT	SP→BG	{ transcript: string }	无 (fire-and-forget)
TTS_SPEAK	BG→SP	{ text: string }	{ done: true }
ANIM_SET_STATE	BG→SP	{ state: 'idle' }	'listening'
GET_API_KEY	BG→BG	无	{ key: string }

4.3 错误消息格式

```
{
  type: 'ERROR',
  payload: {
    source: string,    // 出错模块名
    code: string,      // 错误码 (SELECTOR_NOT_FOUND / API_TIMEOUT / etc.)
    message: string,   // 人类可读错误描述
    recoverable: boolean // 是否可恢复
  }
}
```

```
}  
}
```


五、错误处理策略

5.1 错误即第一公民

每个模块必须显式处理所有可能的失败状态。不允许 try-catch 吞掉错误。

```
// ❌ 错误写法
try { await fetch(url); } catch(e) {}

// ✅ 正确写法
try {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
} catch(e) {
  return { error: true, code: 'API_ERROR', message: e.message, recoverable: true };
}
```

5.2 错误码规范

错误码	模块	含义	恢复策略
DISTILL_EMPTY_DOM	dom-distiller	页面无可交互元素	告知用户当前页面不支持导航
SELECTOR_NOT_FOUND	selector-engine	L1-L3全部失败	触发L4 (LLM重试)
SELECTOR_ALL_FAILED	selector-engine	L1-L4全部失败	触发L5 (语音降级)
API_TIMEOUT	llm-client	DeepSeek超时	重试1次，仍失败则切换模型
API_RATE_LIMIT	llm-client	频率限制	等待后重试
API_KEY_MISSING	llm-client	未配置API Key	提示用户配置
TTS_BROWSER_UNSUPPORTED	tts	浏览器不支持	降级为纯文字显示
ASR_NO_PERMISSION	sidepanel	麦克风权限被拒	降级为文字输入
TASK_TIMEOUT	task-queue	任务步骤超时	跳过当前步骤，继续下一步

5.3 Fail-Fast 原则

错误应立即上报，不静默传播。每个模块的错误返回必须包含完整上下文，让上层决策。

六、状态管理规范

6.1 全局状态（Background SW 维护）

```
const AppState = {
  currentPage: null,          // 当前页面的蒸馏结果
  conversationHistory: [],    // 对话历史（最多保留5轮）
  activeTaskQueue: null,      // 当前执行中的任务队列
  animState: 'idle',          // 当前数字人动画状态
  selectedModel: 'deepseek-chat', // 当前模型
  apiKey: null,                // 从storage同步的API Key
};
```

6.2 状态变更规则

- AppState 只能由 Background SW 修改
- 其他进程通过 Message 请求状态变更
- 状态变更必须记录到 append-only 日志

6.3 对话历史管理

```
// 最多保留5轮对话，超出删除最早记录
const MAX_HISTORY = 5;

function appendConversation(role, content) {
  AppState.conversationHistory.push({ role, content, timestamp: Date.now() });
  if (AppState.conversationHistory.length > MAX_HISTORY) {
    AppState.conversationHistory.shift();
  }
}
```

七、代码规范

7.1 命名规范

类型	规范	示例
文件名	kebab-case	dom-distiller.js
函数名	camelCase	distillDom()
常量	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
消息类型	UPPER_SNAKE_CASE	DOM_DISTILL
CSS 类名	前缀 dh- + kebab-case	.dh-highlight-pulse

7.2 DOM 操作隔离

Content Script 注入的所有 DOM 必须带 dh- 前缀命名空间：

```
/*  正确 */
.dh-highlight-overlay { ... }
.dh-pulse-ripple { ... }

/*  错误 - 会污染宿主页面 */
.highlight { ... }
```

7.3 异步规范

- 所有网络请求使用 async/await
- 所有 Promise 必须有 .catch 处理
- 禁止使用同步 XMLHttpRequest
- setTimeout 必须有清理逻辑

7.4 安全规范

- API Key 禁止硬编码，必须通过 chrome.storage.local 存取
- 用户语音数据禁止持久化存储
- DOM 蒸馏数据只能存在于内存中，不写入磁盘
- Content Script 不能 eval 任何来自 LLM 的代码

八、验证门（Verification Gates）

每个模块完成后，必须通过以下验证才能进入下一模块：

Gate 1: dom-distiller.js

- ☐ 输入空页面 → 返回 `{ regions: {}, elements: [], summary: "页面无内容" }`
- ☐ 输入含 100 个可交互元素的页面 → 蒸馏结果不超过 50 个元素
- ☐ 处理时间 < 50ms (Performance.now 测量)
- ☐ 选择器增强：每个元素至少有 weak + strong + textFallback 三个字段

Gate 2: selector-engine.js

- ☐ L1 精确选择器：在测试页面命中率 100%
- ☐ L2 弱化选择器：在 class 变化 50% 时仍有 80% 命中率
- ☐ L3 XPath：中文按钮文本匹配测试
- ☐ L5 降级：所有级别失败后返回 verbal 模式

Gate 3: llm-client.js

- ☐ DeepSeek Chat 可连接
- ☐ DeepSeek V4 Flash 可连接
- ☐ 返回 JSON 格式正确可解析
- ☐ 超时 10s 自动 reject
- ☐ API Key 缺失时返回明确错误

Gate 4: content.js + content.css

- ☐ 高亮动画不破坏宿主页面布局
- ☐ 注入的 DOM 元素全部带 dh- 前缀
- ☐ MutationObserver 正确触发后续步骤
- ☐ 页面导航（SPA路由变化）后蒸馏自动刷新

Gate 5: end-to-end

- ☐ "帮我点击登录按钮" → 登录按钮被高亮
- ☐ 多步任务：3步任务全部执行
- ☐ 端到端延迟 < 3s

九、子系统的 AGENTS.md 摘要

lib/AGENTS.md

- 此目录下的代码必须是纯函数或无副作用的工具函数
- 不允许 import 浏览器特定API (chrome.*, window.*)
- 所有函数必须有 JSDoc 类型注解
- 每个文件只做一个关注点
- 测试优先级: dom-distiller > selector-engine > task-queue

content/AGENTS.md

- 你是页面操作的唯一执行者
- 所有 DOM 写入必须带 dhn- 前缀命名空间
- 不允许发起网络请求, 全部委托给 Background SW
- CSS 动画用 transform/opacity, 避免触发 reflow
- 错误必须通过 Message 上报, 不能 console.error 后吞掉

background/AGENTS.md

- 你是系统的唯一编排者和状态持有者
- 维护 AppState 的单一数据源
- 所有 API 调用在此发起
- 消息路由必须显式处理, 不允许 fallthrough
- API Key 从 chrome.storage 读取, 不能硬编码

sidepanel/AGENTS.md

- 你是用户界面的唯一渲染者
- 不参与业务逻辑决策
- 只响应 ANIM_SET_STATE 和 TTS_SPEAK 消息
- ASR 结果通过 ASR_RESULT 发送给 Background
- Three.js 场景限制在 Side Panel 的 canvas 内

十、Prompt 模板

DeepSeek 系统 Prompt

你是一个网页导航助手。你会收到一个网页的结构描述和用户的导航请求。
请返回精确的 JSON 指令。

输出格式

```
{
  "target": "CSS选择器",
  "fallbackText": "元素的可见文本(用于XPath降级)",
  "action": "highlight|click|input|scroll",
  "speech": "对用户说的引导语(中文, 20字以内)",
  "tasks": [ // 仅多步任务需要
    {"step": 1, "speech": "...", "target": "...", "trigger":
"click|visible|input"}
  ]
}
```

规则

1. target 必须基于页面结构中实际存在的选择器
2. 不确定的选择器不要编造, 用 fallbackText 标注
3. speech 要简短自然, 像朋友在耳边提示
4. 如果用户请求模糊, 优先选择最可能的元素
5. 多步任务必须按操作顺序排列 steps

蒸馏页面 Prompt 组装

页面类型

{pageType}

页面结构

{regionSummaries}

可交互元素

{elementsList}

用户意图

{userTranscript}

请返回导航指令 JSON。

十一、开发流程约束

11.1 执行顺序

```
Module 1 (dom-distiller.js) → Gate 1 验证
    ↓
Module 2 (selector-engine.js) → Gate 2 验证
    ↓
Module 3 (llm-client.js) → Gate 3 验证
    ↓
Module 4 (content.js + content.css) → Gate 4 验证
    ↓
Module 5 (background/service-worker.js)
    ↓
Module 6 (sidepanel/)
    ↓
Module 7 (task-queue.js)
    ↓
Gate 5 (End-to-End 验证)
```

11.2 提交规范

- 每个 Module 完成后独立提交
- Commit message 格式： [Module N] 简短描述
- 禁止跨模块的大提交

11.3 质量要求

- 每个函数不超过 30 行
- 每个文件不超过 300 行
- 禁止超过 3 层的嵌套
- 禁止魔法数字，必须用命名常量

十二、Harness 债务管理

OpenAI 的实践表明：每个 **harness** 组件都编码了一个关于"模型不能做什么"的假设。当模型升级时，这些假设可能过时。

需要定期审查的假设

假设	何时重新评估
"DeepSeek Chat 不能处理超过 500 token 的 DOM"	模型升级后测试更长的上下文
"选择器降级需要 5 级"	如果 L1 命中率 > 95%，可简化为 3 级
"必需 Side Panel 渲染数字人"	如果性能允许，可考虑注入式渲染
"8B 本地模型不可用"	当 M4 本地推理能力提升后重新评估

Harness 简化原则

每当你发现自己在写一个 **workaround** 时，加一条 `// HARNESS_DEBT` 注释，标注这是为哪个模型的哪个弱点写的。当模型升级时，搜索这些注释并逐一评估是否可以移除。

文件类型：Harness Engineering 约束文件 | 适用项目：数字人网页导航助手 | 版本：v1.0 | 基于：OpenAI Harness Engineering 方法论 (Feb 2026) + Mitchell Hashimoto 实践